

Deep Learning Avancé & Applications

Optimisation Avancée
des Réseaux Neuronaux

Master Bac+5 – Data Analytics & Big Data
Mouaad AGOURRAM

2025–2026

Golden Collar

Ce module suppose acquis

- Architectures de base : ANN, CNN, RNN, LSTM
- Framework : TensorFlow / Keras (niveau intermédiaire)
- Machine Learning : régression, classification, métriques

Programme du module

Séance	Thématique
S1	Optimisation avancée des réseaux neuronaux
S2	Amélioration des performances & généralisation
S3	Architectures CNN avancées
S4	Architectures séquentielles avancées
S5	Transformers & mécanismes d'attention
S6	Deep Learning appliqué au NLP
S7	Deep Learning pour la vision avancée
S8	Interprétabilité, robustesse & éthique
S9	Étude de cas & mini-projet

Plan de la séance

Introduction & Rappels

Fonctions de coût avancées

Optimiseurs avancés

Initialisation des poids

Batch Normalization

Régularisation avancée

Synthèse & TD

Introduction & Rappels

Objectifs pédagogiques de cette séance

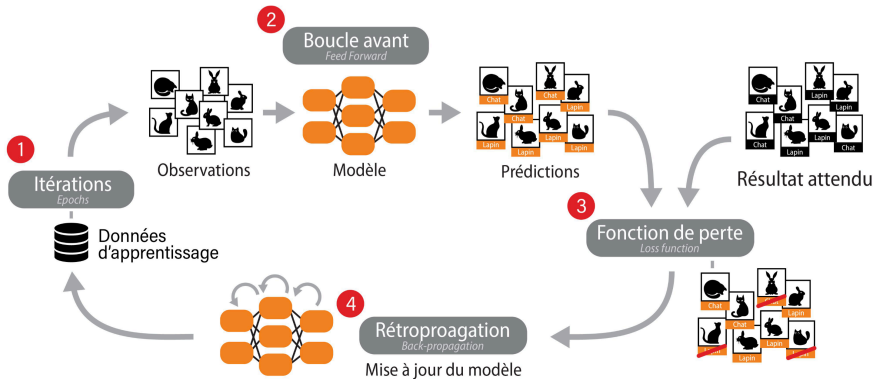
Objectif principal

Comprendre et maîtriser les **techniques d'optimisation avancées** qui permettent d'entraîner efficacement des modèles profonds.

À l'issue de cette séance, vous serez capable de :

- Choisir et implémenter des **fonctions de coût** adaptées au problème
- Utiliser les **optimiseurs modernes** (Adam, AdamW, Muon) avec leurs hyperparamètres
- Appliquer les bonnes pratiques d'**initialisation** et de **normalisation**
- Combiner plusieurs techniques de **régularisation** pour améliorer la généralisation

Rappel : le processus d'entraînement



Chaque composant peut être **optimisé** : choix du loss, de l'optimiseur, initialisation, normalisation, régularisation.

Fonctions de coût avancées

Fonctions classiques

- **MSE** : $\mathcal{L} = \frac{1}{n} \sum (y_i - \hat{y}_i)^2$
- **Cross-Entropy** : $\mathcal{L} = - \sum y_i \log \hat{y}_i$

Limites

- Sensibilité aux outliers (MSE)
- Classes déséquilibrées
- Tâches spécifiques (détection, segmentation)

⇒ Nécessité de fonctions de coût **adaptées au problème**.

Huber Loss (régression robuste)

Formulation

$$\mathcal{L}_\delta(y, \hat{y}) = \begin{cases} \frac{1}{2}(y - \hat{y})^2 & \text{si } |y - \hat{y}| \leq \delta \\ \delta|y - \hat{y}| - \frac{1}{2}\delta^2 & \text{sinon} \end{cases}$$

Caractéristiques

- Combinaison MSE / MAE
- Robuste aux outliers
- δ contrôle la transition

Formulation

$$\mathcal{L}_{FL} = -\alpha_t(1 - p_t)^\gamma \log(p_t)$$

Caractéristiques

- Réduit le poids des exemples faciles
- γ : facteur de focalisation
- α_t : pondération des classes
- Essentiel en détection d'objets

Dice Loss (segmentation)

Formulation

$$\mathcal{L}_{Dice} = 1 - \frac{2|A \cap B|}{|A| + |B|}$$

Caractéristiques

- Mesure le chevauchement
- Efficace pour segmentation binaire
- Gère le déséquilibre spatial

Contrastive Loss (embeddings)

Formulation

$$\mathcal{L} = \frac{1}{2}(1 - y)D^2 + \frac{1}{2}y \max(0, m - D)^2$$

Caractéristiques

- D : distance entre embeddings
- y : paire similaire (0) ou non (1)
- m : marge minimale
- Utilisée en Siamese Networks

Triplet Loss (metric learning)

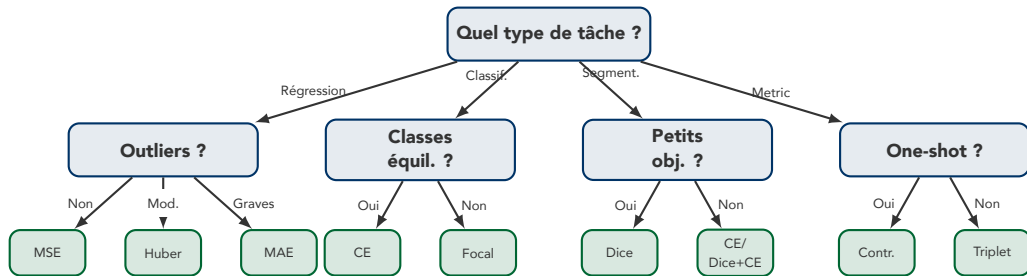
Formulation

$$\mathcal{L} = \max(0, D(a, p) - D(a, n) + m)$$

Caractéristiques

- Triplet : **ancre** a , **positif** p (même classe), **négatif** n
- Rapproche a de p et éloigne a de n d'au moins la marge m
- Généralise la Contrastive Loss (3 points au lieu de 2)
- Le choix des triplets (*hard negative mining*) est déterminant
- Référence : FaceNet (reconnaissance faciale)

Comment choisir sa fonction de coût ? Source : arXiv 2504.04242v1



En pratique

- Commencer simple (MSE, Cross-Entropy)
- Analyser les données (déséquilibre, outliers, taille objets)
- Adapter la loss si performances non satisfaisantes
- Dans certains cas, on peut privilégier une loss personnalisée (métrique métier).

Implémentation en Keras

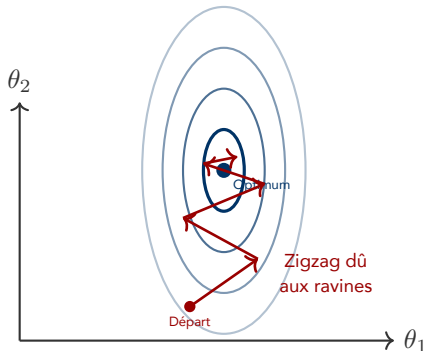
```
1 import tensorflow as tf
2 from tensorflow.keras import backend as K
3
4 # Focal Loss
5 def focal_loss(gamma=2.0, alpha=0.25):
6     def loss_fn(y_true, y_pred):
7         y_pred = tf.clip_by_value(y_pred, 1e-7, 1 - 1e-7)
8         cross_entropy = -y_true * tf.math.log(y_pred)
9         weight = alpha * y_true * tf.pow(1 - y_pred, gamma)
10        return tf.reduce_mean(weight * cross_entropy)
11    return loss_fn
12
13 # Dice Loss
14 def dice_loss(y_true, y_pred, smooth=1.0):
15     intersection = K.sum(y_true * y_pred, axis=-1)
16     union = K.sum(y_true, axis=-1) + K.sum(y_pred, axis=-1)
17     return 1 - (2. * intersection + smooth) / (union + smooth)
18
19 # Utilisation
20 model.compile(optimizer='adam', loss=focal_loss(gamma=2.0, alpha=0.25))
```


Optimiseurs avancés

Limites de SGD classique

$$\text{SGD : } \theta_{t+1} = \theta_t - \eta \nabla \mathcal{L}$$

- Learning rate fixe et global
- Oscillations dans les ravines
- Convergence lente
- Sensible aux minima locaux



⇒ Besoin d'optimiseurs **adaptatifs**.

Momentum et Nesterov (NAG)

Momentum

$$v_t = \mu v_{t-1} + \nabla \mathcal{L}(\theta_t)$$

$$\theta_{t+1} = \theta_t - \eta v_t$$

- Accumule une **inertie** dans les directions cohérentes
- Amortit les oscillations dans les ravines
- μ typique : 0.9

Nesterov (NAG)

$$v_t = \mu v_{t-1} + \nabla \mathcal{L}(\theta_t - \eta \mu v_{t-1})$$

Gradient évalué **après** le pas d'inertie :

Pourquoi c'est central

Le 1^{er} moment m_t d'Adam **est** ce terme v_t . Comprendre Momentum, c'est comprendre la moitié d'Adam.

Cas d'usage

SGD+Momentum reste un **excellent choix** pour les CNN entraînés *from scratch* (ResNet, VGG) : très bonne généralisation.

RMSprop (Root Mean Square Propagation)

Formulation

$$v_t = \beta v_{t-1} + (1 - \beta)(\nabla \mathcal{L})^2$$

$$\theta_{t+1} = \theta_t - \frac{\eta}{\sqrt{v_t} + \epsilon} \nabla \mathcal{L}$$

Caractéristiques

- Adapte le learning rate par paramètre
- Normalise par la moyenne mobile des gradients au carré
- Particulièrement **efficace pour RNN/LSTM**
- Valeur typique : $\beta = 0.9$

Cas d'usage

Privilégier RMSprop pour les architectures récurrentes (RNN, LSTM, GRU)

Adam (Adaptive Moment Estimation)

Principe

Adam combine les avantages de **Momentum** (1er moment) et **RMSprop** (2e moment)

Formulation

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) \nabla \mathcal{L}_t \quad (1\text{er moment})$$

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) (\nabla \mathcal{L}_t)^2 \quad (2\text{e moment})$$

$$\hat{m}_t = \frac{m_t}{1 - \beta_1^t}, \quad \hat{v}_t = \frac{v_t}{1 - \beta_2^t} \quad (\text{correction de biais})$$

$$\theta_{t+1} = \theta_t - \frac{\eta}{\sqrt{\hat{v}_t} + \epsilon} \hat{m}_t$$

Caractéristiques

- Learning rate adaptatif par paramètre
- Valeurs typiques : $\beta_1 = 0.9$, $\beta_2 = 0.999$, $\epsilon = 10^{-8}$
- Convergence rapide
- Bon pour le prototypage

Formulation

$$\theta_{t+1} = \theta_t - \eta \left(\frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon} + \lambda \theta_t \right)$$

Caractéristiques

- Adam + **decoupled weight decay**
- Weight decay découplé de l'optimisation adaptative
- Meilleure généralisation que Adam classique
- **Recommandé** pour les Transformers et architectures modernes
- λ typique : 10^{-2} à 10^{-1}

Différence clé avec Adam

Le weight decay est appliqué **directement** sur les poids, pas via le gradient

Muon – Optimiseur orthogonalisé (2024)

Qu'est-ce que Muon ?

- Optimiseur hybride (2024) combinant :
 - **Momentum orthogonalisé** pour les matrices de poids
 - **AdamW** pour les biais/normalisations
- Proposé par Keller Jordan et al. (2024)
- À l'origine de records de vitesse sur le *speedrun* nanoGPT

Avantages observés

- Jusqu'à $\sim 2x$ moins d'itérations qu'AdamW sur certains benchmarks
- Bonne généralisation
- Moins sensible au learning rate
- Mémoire comparable à SGD+Momentum

Algorithm: Muon

Require: Learning rate η , momentum μ

1. Initialize $B_0 \leftarrow 0$
2. **for** $t = 1, \dots$ **do**
3. Compute gradient $G_t \leftarrow \nabla_{\theta} \mathcal{L}_t(\theta_{t-1})$
4. $B_t \leftarrow \mu B_{t-1} + G_t$
5. $O_t \leftarrow \text{NewtonSchulz5}(B_t)$
6. Update $\theta_t \leftarrow \theta_{t-1} - \eta O_t$
7. **end for**
8. **return** θ_t

NewtonSchulz5 : Orthogonalisation

- **Objectif:** Projeter B_t orthogonalement
- **Méthode:** 5 itérations Newton-Schulz pour approximer $B_t(B_t^T B_t)^{-1/2}$
- **Stabilité:** Évite l'inversion directe (coûteuse et instable)

Comparaison des optimiseurs

Critère	SGD	SGD+Mom	RMSProp	Adam	AdamW	Muon
Mémoire	Très faible	Faible	Moyenne	Élevée	Élevée	Moyenne
Vitesse convergence	Très lente	Lente	Rapide	Rapide	Rapide	Très rapide
Généralisation	Bonne	Excellente	Bonne	Moyenne	Bonne	Excellente
Sensibilité LR	Très élevée	Élevée	Moyenne	Moyenne	Faible	Très faible
Cas d'usage	Rarement utilisé	CV classique	RNNs, RL	RNNs, petits modèles	LLMs, Transformers	LLMs, Vision

Règle pratique

- **Muon** : LLMs, Vision Transformers, modèles $>100M$ params
- **AdamW** : Transformers standards, fine-tuning, modèles $<100M$
- **SGD+Momentum** : CNNs classiques (ResNet), apprentissage from scratch
- **Adam** : prototypage rapide, RNNs, petits modèles.

Initialisation des poids

Pourquoi l'initialisation est critique ?

Objectif : maintenir la variance des activations constante à travers les couches.

Enjeu

L'initialisation des poids détermine la capacité du réseau à apprendre efficacement, surtout pour les architectures profondes.

Risques d'une mauvaise initialisation

- Vanishing ou exploding gradients
- Convergence très lente ou divergence
- Blocage dans des minima locaux

Bénéfices d'une bonne initialisation

- Variance stable, gradients bien propagés
- Convergence rapide et stable

Vanishing Gradients

Poids trop petits $\Rightarrow$ gradients $\rightarrow 0$

- Les couches profondes ne s'entraînent pas
- Convergence extrêmement lente
- Saturation des fonctions d'activation (sigmoïde, tanh)

Mauvaise initialisation : les problèmes

Exploding Gradients

Poids trop grands $\Rightarrow$ gradients $\rightarrow \infty$

- Valeurs numériques instables (NaN, Inf)
- Divergence de l'entraînement
- Oscillations importantes de la loss

Convergence lente

- Initialisation inadaptée à l'architecture
- Nécessité d'un learning rate très faible
- Risque de rester bloqué dans un minimum local

Bonne initialisation : les bénéfices

Variance des activations stable

- Les activations restent dans une plage raisonnable
- Évite la saturation des fonctions d'activation
- Facilite l'apprentissage de toutes les couches

Gradients bien propagés

- Les gradients circulent efficacement dans tout le réseau
- Toutes les couches peuvent apprendre simultanément
- Permet d'entraîner des réseaux très profonds (ResNet, etc.)

Convergence rapide et stable

- L'entraînement converge en moins d'époques
- Trajectoire d'optimisation plus directe
- Meilleure généralisation finale

Formulation

$$W \sim \mathcal{N}\left(0, \frac{2}{n_{in} + n_{out}}\right)$$

où n_{in} = nombre d'entrées, n_{out} = nombre de sorties

Principe

Maintient la variance des activations et des gradients constante en considérant à la fois les entrées et les sorties de la couche.

Caractéristiques

- Adapté aux activations **sigmoïde, tanh**
- Maintient la variance entre couches (forward et backward)
- Par défaut dans Keras (`glorot_uniform`)
- Variante uniforme : $U\left[-\sqrt{\frac{6}{n_{in}+n_{out}}}, \sqrt{\frac{6}{n_{in}+n_{out}}}\right]$

Formulation

$$W \sim \mathcal{N}\left(0, \frac{2}{n_{in}}\right)$$

où n_{in} = nombre d'entrées

Principe

Adapté spécifiquement pour ReLU qui "tue" la moitié des neurones (sortie = 0 pour valeurs négatives), nécessitant une variance plus élevée.

Caractéristiques

- Adapté aux activations **ReLU et variantes** (LeakyReLU, PReLU)
- Compense les neurones "morts" par ReLU
- **Recommandé** pour réseaux profonds modernes (ResNet, VGG, etc.)
- Variante uniforme : $U\left[-\sqrt{\frac{6}{n_{in}}}, \sqrt{\frac{6}{n_{in}}}\right]$

Règle de choix de l'initialisation

Règle simple selon la fonction d'activation

ReLU $\Rightarrow$ **He** | Sigmoid/Tanh $\Rightarrow$ **Xavier** | SELU $\Rightarrow$ **LeCun**

LeCun Initialization

$$W \sim \mathcal{N}\left(0, \frac{1}{n_{in}}\right)$$

Utilisée avec SELU (Self-Normalizing Networks)

Recommandations pratiques

- **Par défaut** : He pour architectures modernes (CNN avec ReLU)
- **Transformers** : Xavier (ou variantes spécifiques comme GPT)
- **RNN/LSTM** : Orthogonal initialization pour matrices récurrentes
- **Transfer Learning** : conserver les poids pré-entraînés !

Implémentation en TensorFlow / Keras

```
1 from tensorflow.keras import layers
2
3 # He initialization (par défaut pour Dense)
4 layers.Dense(128, activation='relu',
5             kernel_initializer='he_normal')
6
7 # Xavier / Glorot initialization
8 layers.Dense(64, activation='tanh',
9             kernel_initializer='glorot_uniform')
10
11 # LeCun initialization
12 layers.Dense(256, activation='selu',
13             kernel_initializer='lecun_normal')
```

Variantes disponibles dans Keras

```
1 # Variantes Glorot (Xavier)
2 'glorot_uniform' # Par défaut pour Dense
3 'glorot_normal'  # Distribution normale
4
5 # Variantes He
6 'he_uniform'     # Distribution uniforme
7 'he_normal'      # Distribution normale (recommande)
8
9 # Variantes LeCun
10 'lecun_uniform'  # Distribution uniforme
11 'lecun_normal'   # Distribution normale
```

Exemple : CNN avec He

```
1 from tensorflow.keras import models, layers
2
3 model = models.Sequential([
4     layers.Conv2D(32, (3,3), activation='relu',
5                   kernel_initializer='he_normal',
6                   input_shape=(28, 28, 1)),
7     layers.MaxPooling2D((2, 2)),
8     layers.Conv2D(64, (3,3), activation='relu',
9                   kernel_initializer='he_normal'),
10    layers.MaxPooling2D((2, 2)),
11    layers.Flatten(),
12    layers.Dense(128, activation='relu',
13                 kernel_initializer='he_normal'),
14    layers.Dense(10, activation='softmax',
15                 kernel_initializer='glorot_uniform')
16 ])
```

Note

ReLU $\Rightarrow$ he_normal | Softmax finale $\Rightarrow$ glorot_uniform

Batch Normalization

Batch Normalization (Ioffe & Szegedy, 2015)

Principe

Normaliser les activations de chaque couche sur le mini-batch :

$$\hat{x}_i = \frac{x_i - \mu_B}{\sqrt{\sigma_B^2 + \epsilon}} \quad \Rightarrow \quad y_i = \gamma \hat{x}_i + \beta$$

Avantages

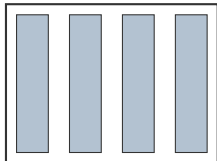
- Réduit le *internal covariate shift*
- Permet des learning rates plus élevés
- Effet régularisant (réduit le besoin de Dropout)
- Accélère la convergence

Points d'attention

- Dépend de la taille du batch
- Comportement différent train/test
- Problématique pour les petits batches
- Alternative : Layer Norm, Group Norm

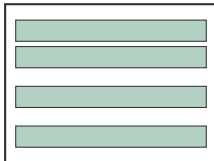
Variantes de normalisation

Batch Norm



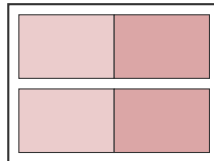
Normalise par feature

Layer Norm



Normalise par exemple

Group Norm



Normalise par groupe

Méthode	Cas d'usage	Dépend du batch ?
Batch Norm	CNN avec grands batches	Oui
Layer Norm	Transformers, RNN	Non
Group Norm	CNN avec petits batches	Non
Instance Norm	Style Transfer	Non

Implémentation : Batch Normalization

Utilisation dans un CNN

```
1 from tensorflow.keras import layers, models
2
3 model = models.Sequential([
4     layers.Conv2D(32, 3, activation='relu',
5                 input_shape=(28, 28, 1)),
6     layers.BatchNormalization(), # Après activation
7     layers.MaxPooling2D(2, 2),
8     layers.Conv2D(64, 3, activation='relu'),
9     layers.BatchNormalization(),
10    layers.MaxPooling2D(2, 2),
11    layers.Flatten(),
12    layers.Dense(128, activation='relu'),
13    layers.BatchNormalization(),
14    layers.Dense(10, activation='softmax')
15 ])
```

Position : un débat

Deux écoles : Dense/Conv → **BN** → activation (convention d'origine, celle de ResNet, la plus répandue) ou bien BN **après** l'activation. À tester ; Conv-BN-ReLU reste le défaut le plus sûr.

Implémentation : Layer Normalization

Utilisation pour RNN / Transformers

```
1 import tensorflow as tf
2 from tensorflow.keras import layers, Model
3
4 # Exemple avec un reseau dense
5 def dense_model(input_shape):
6     inputs = tf.keras.Input(shape=input_shape)
7     x = layers.Dense(128, activation='relu')(inputs)
8     x = layers.LayerNormalization()(x)
9     x = layers.Dense(64, activation='relu')(x)
10    x = layers.LayerNormalization()(x)
11    outputs = layers.Dense(10, activation='softmax')(x)
12    return Model(inputs=inputs, outputs=outputs)
13
14 # Exemple avec LSTM
15 def rnn_model(input_shape):
16     inputs = tf.keras.Input(shape=input_shape)
17     x = layers.LSTM(128, return_sequences=True)(inputs)
18     x = layers.LayerNormalization()(x)
19     x = layers.LSTM(64)(x)
20     outputs = layers.Dense(10, activation='softmax')(x)
21     return Model(inputs=inputs, outputs=outputs)
```

Implémentation : Group Normalization

Avec TensorFlow Addons (petits batchs)

```
1 import tensorflow as tf
2 import tensorflow_addons as tfa
3 from tensorflow.keras import layers, Model
4
5 # Group Normalization pour CNN avec petits batchs
6 def cnn_model():
7     inputs = tf.keras.Input(shape=(224, 224, 3))
8     x = layers.Conv2D(32, 3, activation='relu')(inputs)
9     x = tfa.layers.GroupNormalization(groups=8)(x) # 32/8 = 4 channels par groupe
10    x = layers.Conv2D(64, 3, activation='relu')(x)
11    x = tfa.layers.GroupNormalization(groups=16)(x) # 64/16 = 4 channels par groupe
12    x = layers.Flatten()(x)
13    outputs = layers.Dense(10, activation='softmax')(x)
14    return Model(inputs=inputs, outputs=outputs)
```

Comparaison pratique : BatchNorm vs LayerNorm

Choix selon l'architecture

```
1 import tensorflow as tf
2 from tensorflow.keras import Model, layers
3
4 # CNN classique avec grands batchs (batch_size >= 32)
5 def cnn_model(input_shape):
6     inputs = tf.keras.Input(shape=input_shape)
7     x = layers.Conv2D(64, 3, activation='relu')(inputs)
8     x = layers.BatchNormalization()(x) # Optimal pour CNN
9     x = layers.Conv2D(128, 3, activation='relu')(x)
10    x = layers.BatchNormalization()(x)
11    return Model(inputs=inputs, outputs=x)
12
13 # RNN ou petits batchs (batch_size < 16)
14 def rnn_model(input_shape):
15     inputs = tf.keras.Input(shape=input_shape)
16     x = layers.LSTM(128, return_sequences=True)(inputs)
17     x = layers.LayerNormalization()(x) # Independant du batch
18     x = layers.LSTM(64)(x)
19     x = layers.LayerNormalization()(x)
20    return Model(inputs=inputs, outputs=x)
```

Regle pratique

BatchNorm pour CNN (batch ≥ 32) | LayerNorm pour RNN/Transformers ou petits batchs

Configuration avancée de BatchNormalization

Configuration des paramètres γ et β (scale et shift)

```
1 import tensorflow as tf
2 from tensorflow.keras import Model, layers
3
4 # Créer la couche BN une seule fois
5 bn_layer = layers.BatchNormalization(
6     momentum=0.9,      # Facteur pour moving average (default: 0.99)
7     epsilon=1e-3,      # Stabilité numérique
8     center=True,       # Apprendre le paramètre beta
9     scale=True,        # Apprendre le paramètre gamma
10    trainable=True     # Activer l'apprentissage
11 )
12
13 def model_with_shared_bn(input_shape):
14     inputs = tf.keras.Input(shape=input_shape)
15     x = layers.Conv2D(64, 3, activation='relu')(inputs)
16     x = bn_layer(x)     # Utilise la couche BN préconfigurée
17     outputs = layers.Dense(10, activation='softmax')(x)
18     return Model(inputs=inputs, outputs=outputs)
```

Attention

En mode inference, BatchNorm utilise les statistiques moyennes (non celles du batch)

Régularisation avancée

Régularisation : de quoi parle-t-on ?

Objectif

Réduire l'écart entre performance d'entraînement et de test (**généralisation**), sans trop augmenter le biais.

Renvoi – Séance 2

Le **diagnostic** de l'overfitting (courbes train/val, biais-variance), le **Dropout**, l'**early stopping** et le **learning rate scheduling** sont détaillés en Séance 2. Ici : les leviers intégrés à l'optimisation.

Leviers couverts dans cette section

Weight decay (L2 découplée) | Label smoothing | Data augmentation | Gradient clipping

Weight Decay (L2 découplée)

Formulation

$$\theta_{t+1} = (1 - \lambda\eta)\theta_t - \eta\nabla\mathcal{L}$$

Principe

La régularisation L2 découplée sépare la pénalisation des poids de la mise à jour par gradient, permettant un contrôle indépendant de ces deux mécanismes.

Weight Decay (L2 découplée)

Caractéristiques

- Pénalise les grands poids pour éviter l'overfitting
- Implémentation découplée dans AdamW (appliquée directement sur θ)
- λ typique : 10^{-4} à 10^{-2}
- Différent de L2 régularisation classique (couplée au gradient)

Utilisation

Privilégier AdamW avec weight decay découplé plutôt qu'Adam avec L2 classique

Formulation

$$y'_i = (1 - \epsilon)y_i + \frac{\epsilon}{K}$$

où K est le nombre de classes, ϵ le facteur de smoothing

Principe

Au lieu d'utiliser des labels one-hot stricts (0 ou 1), on adoucit les cibles en redistribuant une petite probabilité ϵ/K sur toutes les classes.

Caractéristiques

- Évite la surconfiance du modèle (probabilités trop proches de 0 ou 1)
- ϵ typique : 0.1 (10% de smoothing)
- Améliore la calibration des probabilités prédites
- Régularisation implicite qui améliore la généralisation

Exemple

Pour $K = 10$ classes et $\epsilon = 0.1$: vraie classe $\rightarrow 0.9$, autres $\rightarrow 0.01$

Data Augmentation

Principe

Augmenter artificiellement la taille et la diversité du dataset en appliquant des transformations aux données d'entraînement.

Techniques classiques (images)

- **Transformations géométriques** : rotations, flips, crops, zoom
- **Transformations colorimétriques** : brightness, contrast, saturation
- **Mixup** : $\tilde{x} = \lambda x_i + (1 - \lambda)x_j$ (mélange de deux images)
- **CutMix** : remplace une région d'une image par une région d'une autre
- **CutOut** : masque aléatoire de régions rectangulaires
- **RandAugment** : sélection aléatoire de transformations

Impact

Améliore significativement la généralisation, surtout avec peu de données

Formulation

$$\nabla \mathcal{L} \leftarrow \frac{\nabla \mathcal{L}}{\max \left(1, \frac{\|\nabla \mathcal{L}\|}{c} \right)}$$

où c est le seuil maximum de la norme du gradient

Principe

Limiter la norme du gradient pour éviter les mises à jour trop importantes qui peuvent déstabiliser l'entraînement.

Caractéristiques

- Préviend l'explosion des gradients ($\text{gradients} \rightarrow \infty$)
- Essentiel pour RNN / LSTM (problème du vanishing/exploding gradient)
- c typique : 1.0 à 5.0
- Deux variantes : clipping par norme globale ou par valeur

Cas d'usage

Implémentation : Weight Decay avec AdamW

Utilisation d'AdamW dans TensorFlow/Keras

```
1 import tensorflow as tf
2 from tensorflow.keras import layers, Model, optimizers
3
4 def create_model(input_shape, num_classes=10):
5     inputs = tf.keras.Input(shape=input_shape)
6     x = layers.Dense(128, activation='relu')(inputs)
7     x = layers.Dense(64, activation='relu')(x)
8     outputs = layers.Dense(num_classes, activation='softmax')(x)
9     return Model(inputs=inputs, outputs=outputs)
10
11 model = create_model(input_shape=(784,), num_classes=10)
12
13 # AdamW avec weight decay decouple
14 optimizer = optimizers.AdamW(
15     learning_rate=0.001,
16     weight_decay=0.01, # Lambda typique
17     beta_1=0.9,
18     beta_2=0.999
19 )
20
21 model.compile(optimizer=optimizer,
22               loss='categorical_crossentropy',
23               metrics=['accuracy'])
```

Note

AdamW est disponible depuis TensorFlow 2.14+ (sinon utiliser `tfa.optimizers.AdamW`)

Implémentation : Label Smoothing

Intégré dans la fonction de perte

```
1  # Option 1 : directement dans la loss
2  loss = tf.keras.losses.CategoricalCrossentropy(
3      label_smoothing=0.1  # Epsilon = 10%
4  )
5  model.compile(optimizer='adam', loss=loss)
6
7  # Option 2 : avec from_logits
8  loss = tf.keras.losses.CategoricalCrossentropy(
9      from_logits=True,
10     label_smoothing=0.1
11 )
```

Remarque

Utiliser `from_logits=True` pour la stabilité numérique

Implémentation : Data Augmentation

Avec Keras preprocessing layers

```
1 import tensorflow as tf
2 from tensorflow.keras import layers, Model
3
4 # Créer les couches d'augmentation
5 def create_data_augmentation():
6     return tf.keras.Sequential([
7         layers.RandomFlip("horizontal"),
8         layers.RandomRotation(0.1),
9         layers.RandomZoom(0.1),          # +/- 10%
10        layers.RandomContrast(0.1),
11    ], name='data_augmentation')
12
13 # Integration via l'API fonctionnelle
14 def create_model(input_shape, num_classes=10):
15     inputs = tf.keras.Input(shape=input_shape)
16     x = create_data_augmentation()(inputs)
17     # ... architecture CNN ...
18     return Model(inputs=inputs, outputs=x)
```

Implémentation : Gradient Clipping

Configuration dans l'optimiseur

```
1 import tensorflow as tf
2
3 # Option 1 : clipping par norme globale
4 optimizer = tf.keras.optimizers.Adam(
5     learning_rate=0.001,
6     clipnorm=1.0      # Norme maximale du gradient
7 )
8
9 # Option 2 : clipping par valeur
10 optimizer = tf.keras.optimizers.Adam(
11     learning_rate=0.001,
12     clipvalue=0.5     # Valeur max de chaque composante
13 )
14
15 model.compile(optimizer=optimizer, loss='mse')
```

Recommandation

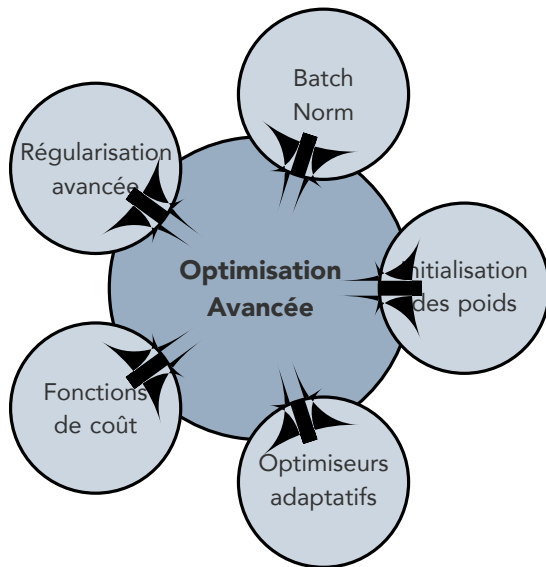
Utiliser clipnorm pour RNN/LSTM, typiquement entre 1.0 et 5.0

Exemple complet : Regularisation combinee

```
1 import tensorflow as tf
2 from tensorflow.keras import layers, Model, regularizers
3
4 # Data Augmentation
5 def create_augmentation():
6     return tf.keras.Sequential([
7         layers.RandomFlip("horizontal"),
8         layers.RandomRotation(0.15),
9     ], name='augmentation')
10
11 # Modele avec regularisation L2 sur les poids
12 def create_model(input_shape, num_classes=10):
13     inputs = tf.keras.Input(shape=input_shape)
14     x = create_augmentation()(inputs)
15     x = layers.Conv2D(
16         32, 3, activation='relu',
17         kernel_regularizer=regularizers.L2(0.001)
18     )(x)
19     x = layers.BatchNormalization()(x)
20     x = layers.Dropout(0.3)(x)
21     x = layers.Flatten()(x)
22     outputs = layers.Dense(num_classes,
23                             activation='softmax')(x)
24     return Model(inputs=inputs, outputs=outputs)
```

```
1 # Optimiseur avec gradient clipping
2 optimizer = tf.keras.optimizers.AdamW(
3     learning_rate=0.001,
4     weight_decay=0.01,
5     clipnorm=1.0
6 )
7
8 # Loss avec label smoothing
9 loss = tf.keras.losses.CategoricalCrossentropy(
10     from_logits=False,
11     label_smoothing=0.1
12 )
13
14 # Compilation
15 model.compile(
16     optimizer=optimizer,
17     loss=loss,
18     metrics=['accuracy']
19 )
20
21 model.summary()
```

Synthèse & TD



Ce qu'il faut retenir pour le TP

Configuration recommandée pour un CNN moderne

1. **Init** : He (`he_normal`) pour les couches ReLU
2. **Normalisation** : BatchNorm (`batch` ≥ 32), sinon GroupNorm / LayerNorm
3. **Optimiseur** : AdamW (Transformers, prototypage) ou SGD+Momentum (CNN *from scratch*)
4. **Régularisation** : weight decay 10^{-4} – 10^{-2} + label smoothing 0.1
5. **Stabilité** : gradient clipping (`clipnorm` 1–5) si RNN/LSTM
6. **Loss** : adaptée au problème (cf. arbre de décision)

Méthode

Partir de cette base, puis **changer un seul levier à la fois** et mesurer (cf. Séance 2).

TD : Analyse comparative d'optimiseurs

- Entraîner un MLP sur MNIST avec SGD, Adam, AdamW
- Comparer les courbes de convergence
- Étudier l'impact de l'initialisation (Xavier vs He)

TP : Optimisation d'un CNN sur CIFAR-10

- Implémenter une Focal Loss personnalisée
- Tester BatchNorm vs LayerNorm vs sans normalisation
- Combiner : AdamW + He init + BatchNorm + Label Smoothing
- Analyser et comparer les résultats

Questions ?

Séance suivante : *Amélioration des performances & généralisation*